Edge Routing Inside a Channel

Anonymous author(s)
Anonymous affiliation(s)

Abstract

We present an efficient edge routing algorithm that produces the shortest path in the homotopy class of an initial polyline route. The key idea is *routing inside a channel*: given an initial path routed on a spanner graph, we construct a polygon (the channel) around the path using a Constrained Delaunay Triangulation, then apply the funnel algorithm to find the optimal path within that channel. Unlike prior approaches, our method does not require the channel polygon to be simple. We implement the algorithm in two open-source graph layout libraries, MSAGLJS (TypeScript) and MSAGL (C#), and demonstrate its application to both force-directed and Sugiyama (layered) layouts. Experimental results show the method produces visually superior paths with competitive performance.

2012 ACM Subject Classification Human-centered computing → Graph drawings; Theory of computation → Computational geometry

Keywords and phrases Edge routing, Constrained Delaunay Triangulation, funnel algorithm, Sugiyama layout, graph drawing

Category Track 2, Short Paper

Generative AI Declaration Generative AI was used for editorial assistance in the preparation of this article.

1 Introduction

A common approach to edge routing in graph drawing is to find shortest paths on a visibility graph that avoids node obstacles. However, the full visibility graph can have $O(n^2)$ edges, making this expensive for large graphs. A practical alternative, introduced by Dwyer and Nachmanson [13], routes edges on a spanner graph—a sparse approximation of the visibility graph—and then applies local optimizations to shorten the resulting paths.

We observe that the local shortcutting heuristic used in [13] can fail to remove all kinks from a path. We replace it with a more principled optimization: for each edge, we construct a *channel*—a polygon surrounding the initial path—and apply the funnel algorithm [9, 16] to find the shortest path within the channel's homotopy class.

The funnel algorithm has been used for edge routing before: Graphviz applies it in the Sugiyama layout where a simple containing polygon is readily available [11]. Our contribution is showing how to construct the channel for *any* layout and demonstrating that the funnel algorithm works correctly even when the channel polygon is not simple.

We implement this approach in two open-source libraries: MSAGLJS[1] (TypeScript) and MSAGL[2] (C#), applying it to both force-directed and Sugiyama layouts.

2 Related Work

Graphviz [4] builds the full visibility graph for edge routing in force-directed layouts, which can be slow for large graphs. For Sugiyama layouts, it uses the funnel algorithm [11] but

[1] `https://github.com/microsoft/msagljs`
[2] `https://github.com/microsoft/automatic-graph-layout`

© Anonymous author(s);
licensed under Creative Commons License CC-BY 4.0
34th International Symposium on Graph Drawing and Network Visualization (GD 2026).
Editors: TBD; Article No. 0; pp. 0:1–0:6
Leibniz International Proceedings in Informatics
LIPICS Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl Publishing, Germany

notes the requirement for a simple polygon. yWorks [7] offers "Organic edge routing" based on force-directed simulation, running in $O((n+m)\log(n+m))$ time. The approach of [13] uses a Yao-graph spanner with local shortcutting, which we improve upon.

3 Edge Routing Inside a Channel

3.1 Building the Spanner

The edge routing starts by building a spanner graph, an approximation of the full visibility graph, following [13]. The spanner is built on a variation of a Yao graph [14, 18]. Each node is covered by a polygon with at most 8 corners, which serve as vertices of the spanner. For each vertex, a family of k cones, with angle $\frac{2\pi}{k}$, $k = 12$ in our setting, partitions the plane; for each cone, at most one edge is created to the closest vertex inside it. The resulting spanner has $O(N)$ edges, where N is the number of graph nodes.

3.2 Channel Construction

Before routing edges, we compute a Constrained Delaunay Triangulation (CDT) $\mathcal{T}$ on the set of obstacle polygons $\mathcal{O}$ [10, 12].

For each graph edge, we route an initial path $\mathcal{L}$ on the spanner. Let $\mathcal{S}$ and $\mathcal{E}$ be the obstacles containing the start and end points of $\mathcal{L}$. We define $\mathcal{U}$, the set of all triangles $t \in \mathcal{T}$ such that either $t \subset \mathcal{S} \cup \mathcal{E}$, or t intersects $\mathcal{L}$ and is not inside any obstacle in $\mathcal{O} \setminus \{\mathcal{S}, \mathcal{E}\}$. The union of $\mathcal{U}$ forms polygon $\mathcal{P}$—the *channel*. The boundary of $\mathcal{P}$ consists of all triangle sides that belong to exactly one triangle in $\mathcal{U}$.

3.3 Sleeve and Funnel Optimization

To apply the funnel algorithm, we need a *sleeve*: a sequence of triangles from start to end where consecutive triangles share a side. Since $\mathcal{U}$ may not have this property, we compute a new Constrained Delaunay Triangulation of $\mathcal{P}$ with the boundary edges as constraints, trace $\mathcal{L}$ through it, and extract the sleeve. The funnel algorithm then produces the shortest path in the homotopy class of $\mathcal{L}$.

Crucially, $\mathcal{P}$ need not be simple. The funnel algorithm requires only a valid sleeve, not a simple polygon. When $\mathcal{P}$ self-intersects, the resulting path is the shortest within the sleeve's homotopy class, though not necessarily the globally shortest path inside $\mathcal{P}$ (see Fig. 1).

3.4 Application to Sugiyama Layout

The channel-based routing naturally extends to Sugiyama (layered) layouts. In a Sugiyama layout, edges already follow polyline paths through the layers. These polylines serve as the initial paths $\mathcal{L}$, and the same CDT-based channel construction and funnel optimization apply. This generalizes the Graphviz approach [11], which requires a simple containing polygon that is available only in specific configurations.

Add Sugiyama-specific details and figures after implementation.

4 Direct Routing on the CDT

The channel-based approach of Section 3 requires two steps: an initial route on the spanner graph, followed by channel construction and funnel optimization. We now describe an

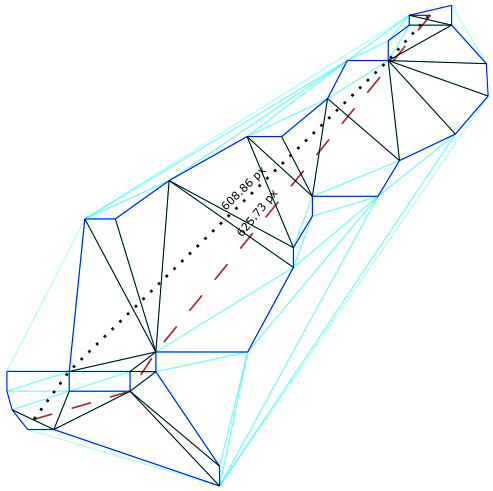

Figure 1 When $\mathcal{P}$ is not simple, the dotted path is shorter than the dashed path found by the funnel algorithm. The funnel result is still the shortest in its homotopy class.

alternative that eliminates the spanner entirely, routing edges directly on the CDT dual graph.

4.1 A* Search on the Dual Graph

Given the CDT $\mathcal{T}$ built on padded obstacle polylines, we define a *dual graph* whose nodes are the CDT triangles and whose edges connect triangles sharing a side. For each graph edge from source s to target t, we locate the triangle containing s and run A* [15] on this dual graph to find a shortest sequence of triangles—the *sleeve*—leading to the triangle containing t.

The A* edge weight between adjacent triangles is the Euclidean distance between their centroids, and the heuristic is the straight-line distance from the current centroid to t. A triangle is excluded from the search if all three of its CDT sites belong to the same obstacle, meaning the triangle lies inside an obstacle interior. Triangles inside the source or target obstacles are permitted, since the path must traverse those obstacles.

The sleeve produced by A* is then passed to the funnel algorithm [9], which computes the shortest path through the sleeve's sequence of diagonals.

4.2 Legal Vertex Movement at Source and Target

A subtlety arises at the source and target nodes. The sleeve passes through triangles whose vertices lie on the padded obstacle boundaries of the source node S and target node T. When these boundary vertices appear as diagonal endpoints, the funnel algorithm threads the path through narrow gaps between obstacle corners, producing sharp zigzag turns near each endpoint.

We eliminate these artifacts by moving each source/target obstacle vertex toward the node center as far as possible without flipping any sleeve triangle's orientation. For a vertex v on the source obstacle boundary, we parametrize its movement as $v' = v + t(s - v)$ for $t \in [0, 1]$. Each sleeve triangle (v, a, b) using v constrains t: defining $c_0 = (b-a) \times (v-a)$ and $c_1 = (b-a) \times (s-v)$, where $\times$ denotes the cross product, the orientation is preserved when $c_0 + t \cdot c_1 > 0$. If $c_1 \geq 0$ the constraint is always satisfied; if $c_1 < 0$ we require $t < c_0/|c_1|$.

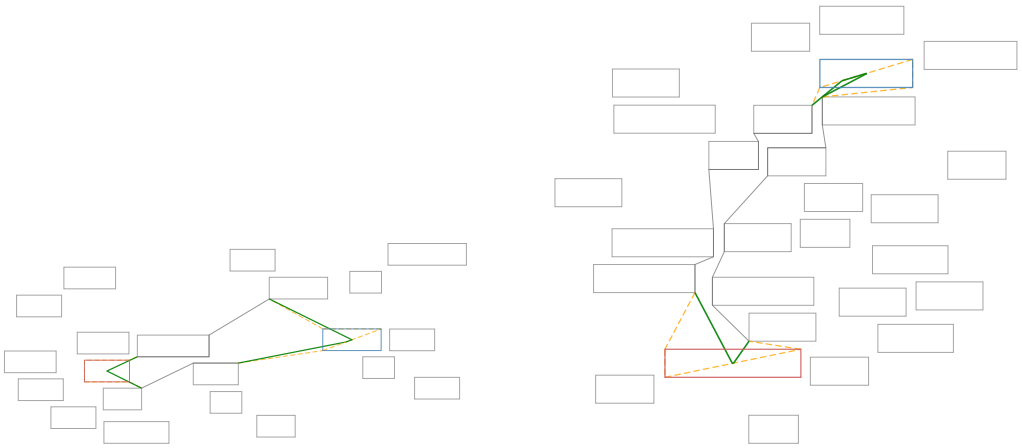

Figure 2 Legal vertex movement on two edges from the Game of Thrones graph. Thin gray: unchanged sleeve boundary. Dashed orange: original position of changed boundary edges. Solid green: boundary edges after moving source/target obstacle vertices toward the node center. Source padded boundary: red. Target padded boundary: blue. Gray rectangles: nearby padded obstacles.

The maximum legal movement is $t^* = \min$ of all upper bounds across adjacent triangles, clamped to $[0, 1]$.

This computation is $O(k)$ per vertex, where k is the number of adjacent sleeve triangles. The result is that each obstacle vertex moves toward the node center as far as the local geometry permits, widening the funnel near endpoints and producing smoother paths. Vertices far from other obstacles typically move all the way to the center, while those in tight spaces move less.

Fig. 2 shows the result on two edges from the Game of Thrones social network. The original sleeve triangles are shown as dashed blue outlines, with the raw diagonals in dashed orange. The red polyline is the funnel path computed after the legal vertex movement.

A final arrowhead-trimming step clips the resulting path at the actual node boundary curves, so the visible edge begins and ends cleanly at each node's perimeter.

4.3 Batched Routing with Dijkstra Trees

When a graph node has many incident edges, running an independent A* search for each edge is wasteful: all searches start from the same source triangle and explore overlapping regions of the CDT. We exploit this by grouping edges by source node and running a single Dijkstra shortest-path computation per source.

For a source node with d outgoing edges, we locate the source triangle and run Dijkstra on the CDT dual graph, expanding outward until all d target triangles have been reached. The Dijkstra builds a shortest-path tree rooted at the source triangle; sleeves to individual targets are recovered by following parent pointers from each target triangle back to the root. Target triangles that lie inside their obstacle's interior are reached but not expanded through, ensuring paths do not traverse non-target obstacles.

This reduces the number of shortest-path searches from m (one per edge) to at most n (one per node), where n and m are the numbers of nodes and edges. In practice the reduction is significant: on the Game of Thrones social network, 2 639 edges originate from only 331 distinct source nodes, with the top source having 93 edges. The Dijkstra tree approach

Graph	Nodes	Edges	Old (s)	New (s)
social network [8]	407	2 639	1.0	1.4
b103 [2]	944	2 438	1.6	2.0
b100 [1]	1 463	5 806	5.6	5.8
composers [6]	3 405	13 832	510.5	20.3
p2p-Gnutella04 [5]	10 876	39 994	375.4	304.2
facebook_combined [3]	4 039	88 234	132.2	123.7
ca-HepPh [17]	12 008	237 010	521.2	495.0

Table 1 Performance comparison (time in seconds).

reduces routing time by 30% compared to per-edge A* on this graph.

The trade-off is that Dijkstra, lacking a target-directed heuristic, may explore more triangles than A* would for any single edge. However, the shortest paths found are identical, since A* with a consistent heuristic and Dijkstra produce the same shortest-path tree.

4.4 Comparison with the Channel Approach

The direct CDT routing avoids building a spanner graph entirely, reducing both code complexity and runtime for the initial routing step. The A* search on the dual graph takes $O(k \log k)$ time per edge, where k is the number of triangles visited. Since the CDT has $O(N)$ triangles for N obstacle vertices, the total routing cost is $O(m \cdot N \log N)$ in the worst case for m edges, though in practice the A* heuristic keeps the search local.

We measured the total edge length produced by each method on the same node layout. On two benchmark graphs—a 407-node social network (2 639 edges) and a 1 463-node graph from the Graphviz test suite—the corridor method produces paths only 2.7–2.8% longer than the spanner-based method, while being significantly simpler to implement.

5 Experimental Results

Update with results from both MSAGLJS and MSAGL .NET implementations, including Sugiyama layouts.

We compare the channel-based routing against the shortcutting approach of [13]. Table 1 summarizes performance on benchmark graphs.

The channel method is faster on larger graphs (over 2 000 nodes) and produces visually superior paths without kinks. On smaller graphs the overhead of CDT construction causes a minor slowdown.

6 Conclusion

We presented an edge routing method based on constructing a channel around an initial path and applying the funnel algorithm to find the shortest path in its homotopy class. The method does not require the channel polygon to be simple, making it applicable to any graph layout. We demonstrated its effectiveness in both force-directed and Sugiyama layouts, implemented in the open-source MSAGLJS and MSAGL libraries.

Finalize after Sugiyama implementation is complete.

References

1. b100. https://github.com/microsoft/automatic-graph-layout/blob/master/GraphLayout/Test/MSAGLTests/Resources/DotFiles/LevFiles/b100.dot.
2. b103. https://github.com/microsoft/automatic-graph-layout/blob/master/GraphLayout/Test/MSAGLTests/Resources/DotFiles/LevFiles/b103.dot.
3. facebookcombined. https://snap.stanford.edu/data/facebook_combined.txt.gz.
4. Graphviz. http://www.graphviz.org/.
5. p2p-gnutella04. https://snap.stanford.edu/data/p2p-Gnutella04.html.
6. Skewed. http://mozart.diei.unipg.it/gdcontest/contest2011/composers.xml.
7. yworks. https://yworks.com/products/yed.
8. Andrew Beveridge and Michael Chemers. The game of game of thrones: Networked concordances and fractal dramaturgy. In *Reading Contemporary Serial Television Universes*, pages 201–225. Routledge, 2018.
9. Bernard Chazelle. A theorem on polygon cutting with applications. In *23rd Annual Symposium on Foundations of Computer Science (sfcs 1982)*, pages 339–349. IEEE, 1982.
10. Boris Delaunay et al. Sur la sphere vide. *Izv. Akad. Nauk SSSR, Otdelenie Matematicheskii i Estestvennyka Nauk*, 7(1):793–800, 1934.
11. David P Dobkin, Emden R Gansner, Eleftherios Koutsofios, and Stephen C North. Implementing a general-purpose edge router. In *Graph Drawing: 5th International Symposium, GD'97 Rome, Italy, September 18–20, 1997 Proceedings 5*, pages 262–271. Springer, 1997.
12. Vid Domiter and Borut Žalik. Sweep-line algorithm for constrained delaunay triangulation. *International Journal of Geographical Information Science*, 22(4):449–462, 2008.
13. Tim Dwyer and Lev Nachmanson. Fast edge-routing for large graphs. In *Graph Drawing: 17th International Symposium, GD 2009, Chicago, IL, USA, September 22-25, 2009. Revised Papers 17*, pages 147–158. Springer, 2010.
14. BE Flinchbaugh and LK Jones. Strong connectivity in directional nearest-neighbor graphs. *SIAM Journal on Algebraic Discrete Methods*, 2(4):461–463, 1981.
15. Peter E. Hart, Nils J. Nilsson, and Bertram Raphael. A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2):100–107, 1968. doi:10.1109/TSSC.1968.300136.
16. John Hershberger and Jack Snoeyink. Computing minimum length paths of a given homotopy class. *Computational geometry*, 4(2):63–97, 1994.
17. Jure Leskovec, Jon Kleinberg, and Christos Faloutsos. Graph evolution: Densification and shrinking diameters. *ACM transactions on Knowledge Discovery from Data (TKDD)*, 1(1):2–es, 2007.
18. Andrew Chi-Chih Yao. On constructing minimum spanning trees in k-dimensional spaces and related problems. *SIAM Journal on Computing*, 11(4):721–736, 1982.